

How the customer website, the API, and four local services fit together.

Everything below is drawn from the actual code in `D:\eCommerce` — the Next.js website, the Fastify API, the Postgres schema, and the auth / cart / checkout request paths that connect them. No AWS, no external services: four Docker containers do the whole backend estate.

Website — **Next.js 15** App Router

API — **Fastify 5** + TypeScript

ORM — **Drizzle** → PostgreSQL 16

State — **Zustand** (persisted)

Auth — **JWT** + rotating refresh + email OTP

Payments — **mock provider**, Razorpay/Stripe-ready

Two deployables, four local services

The website and the API are separate processes on purpose — the API is the only thing that talks to the database, so a future mobile app or admin portal can call the exact same /v1 endpoints without the website being involved at all.

WEBSITE — PORT 3000

Next.js 15, App Router

Server-rendered for SEO on catalogue pages; client components for anything personal (cart, checkout, account). No server-side session — the browser holds the JWT and the guest-cart cookie.

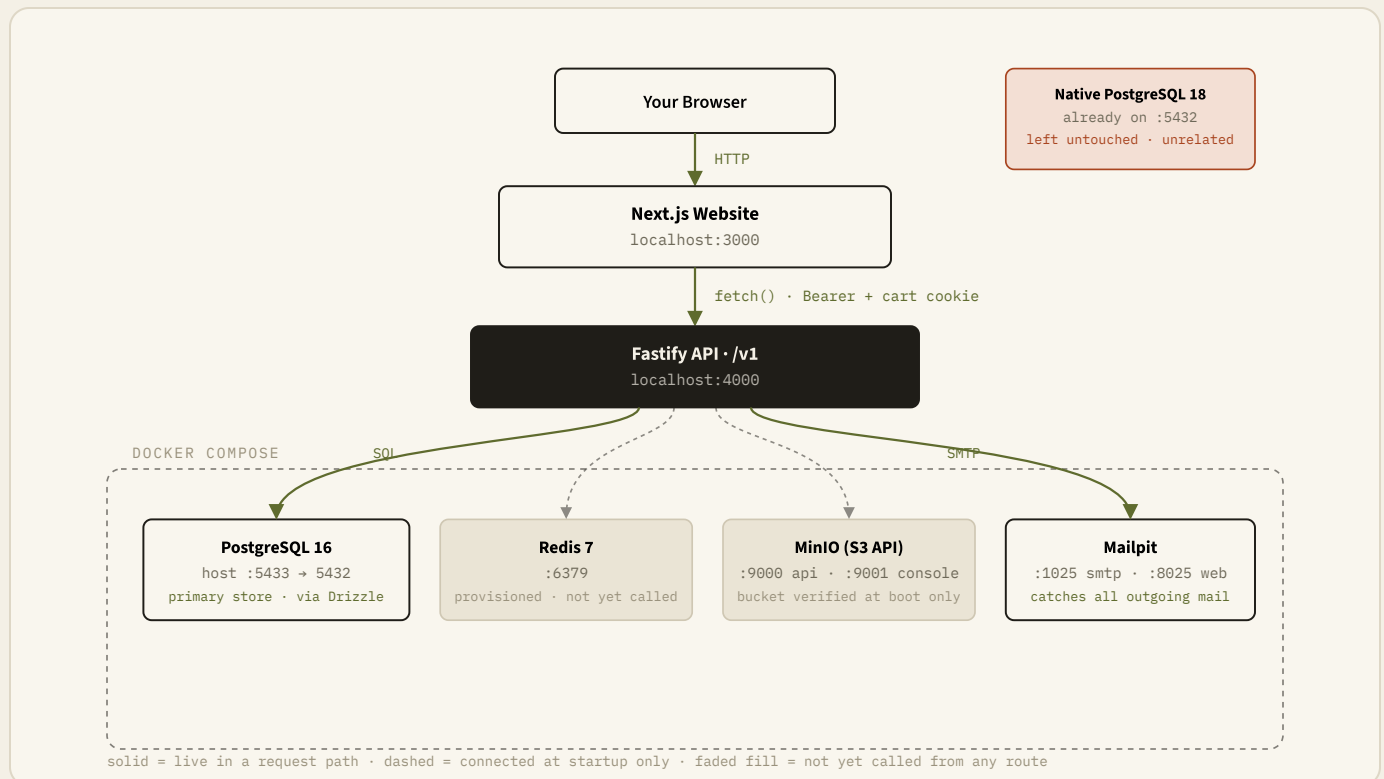
API — PORT 4000

Fastify 5, TypeScript

Stateless REST API under /v1. Every route is reachable by bearer token — the same contract a mobile client or admin portal would use later.

Local topology

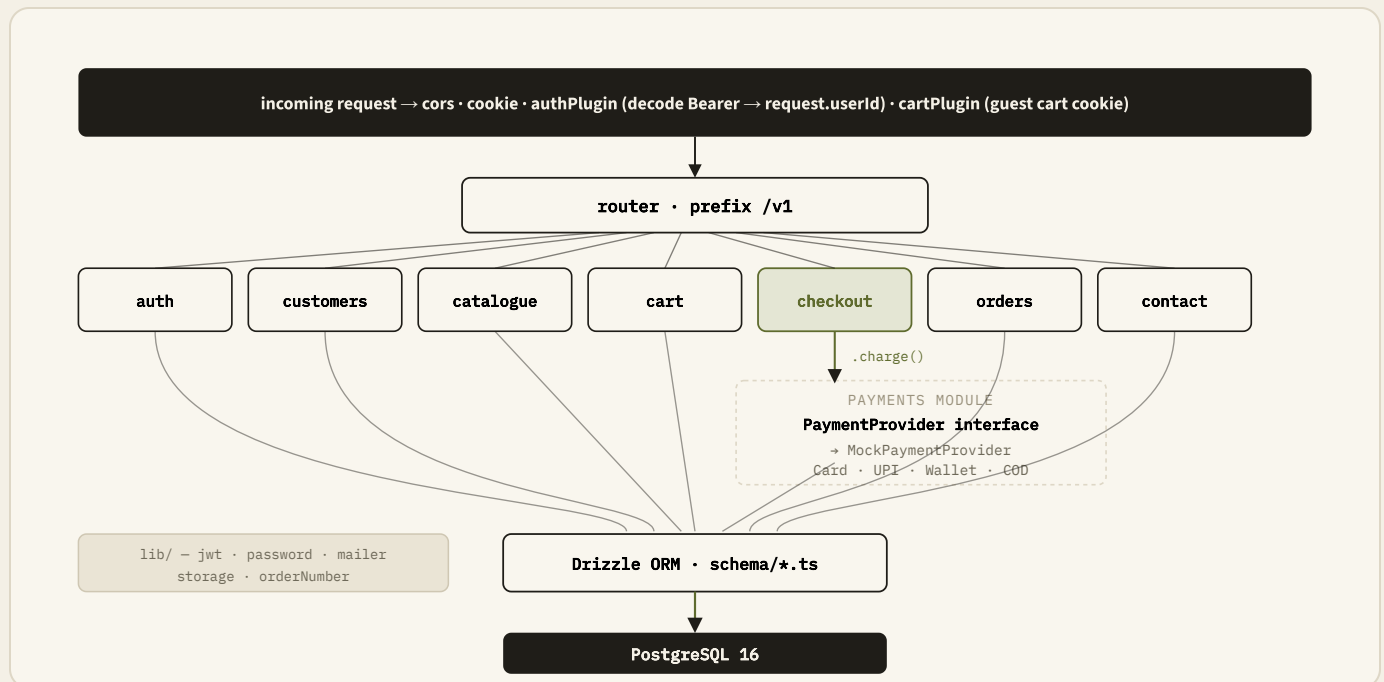
Docker Compose runs four services. Two of them are fully wired into request paths today; two are provisioned ahead of need and not yet called by any route — the diagram marks that distinction rather than implying more than the code does.



The website never talks to Postgres, Redis, MinIO or Mailpit directly — everything routes through the API. Redis and MinIO are running and reachable (the API pings MinIO to verify its bucket on boot) but Phase 1's JWT-based auth and placeholder product imagery mean no request handler calls either of them yet.

Fastify: plugins → routes → services → Drizzle

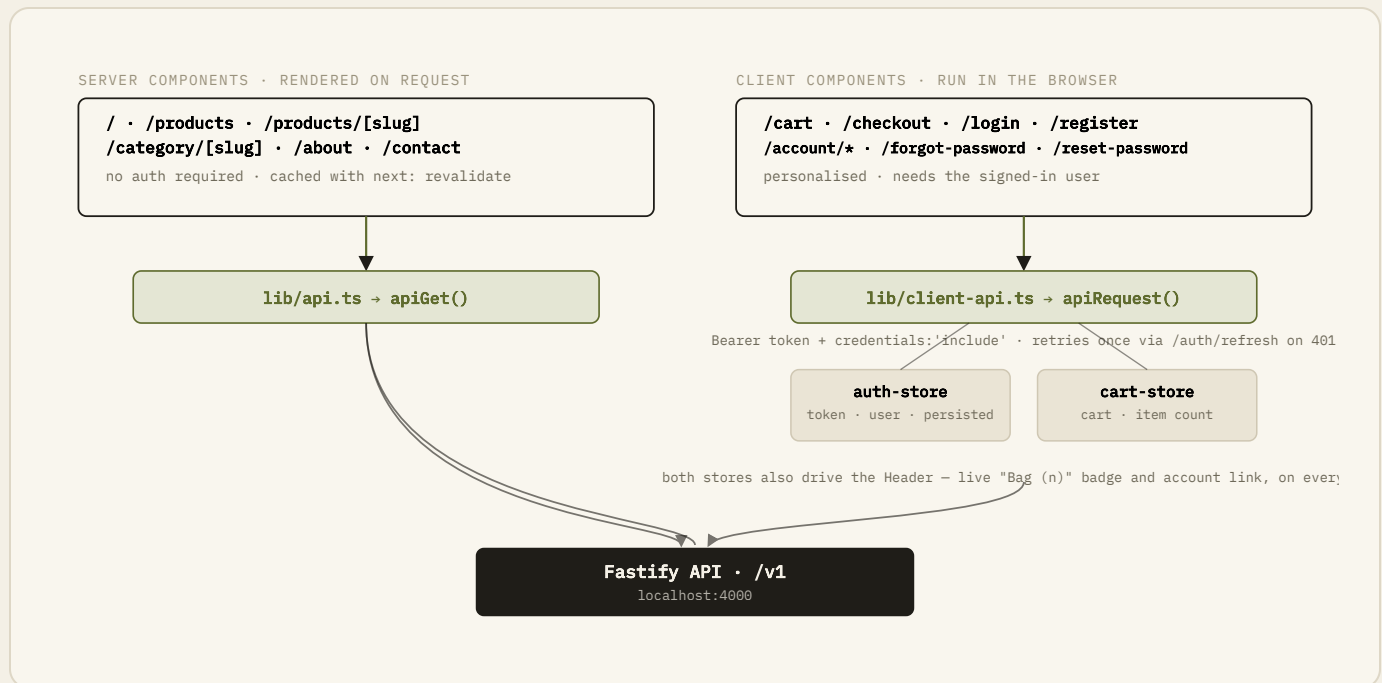
`app.ts` registers two request-scoped plugins before any route runs, then mounts eight route modules under one `/v1` prefix. Every module follows the same shape: a thin `routes.ts` parses input with Zod and calls a `service.ts` that owns the actual logic.



`checkout/service.ts` is the only caller of the payments module — it depends on the `PaymentProvider interface`, not on the mock implementation directly, so a real Razorpay/Stripe adapter drops in behind `registry.ts` with zero changes to checkout logic.

Two fetch paths: server-rendered vs. client-authenticated

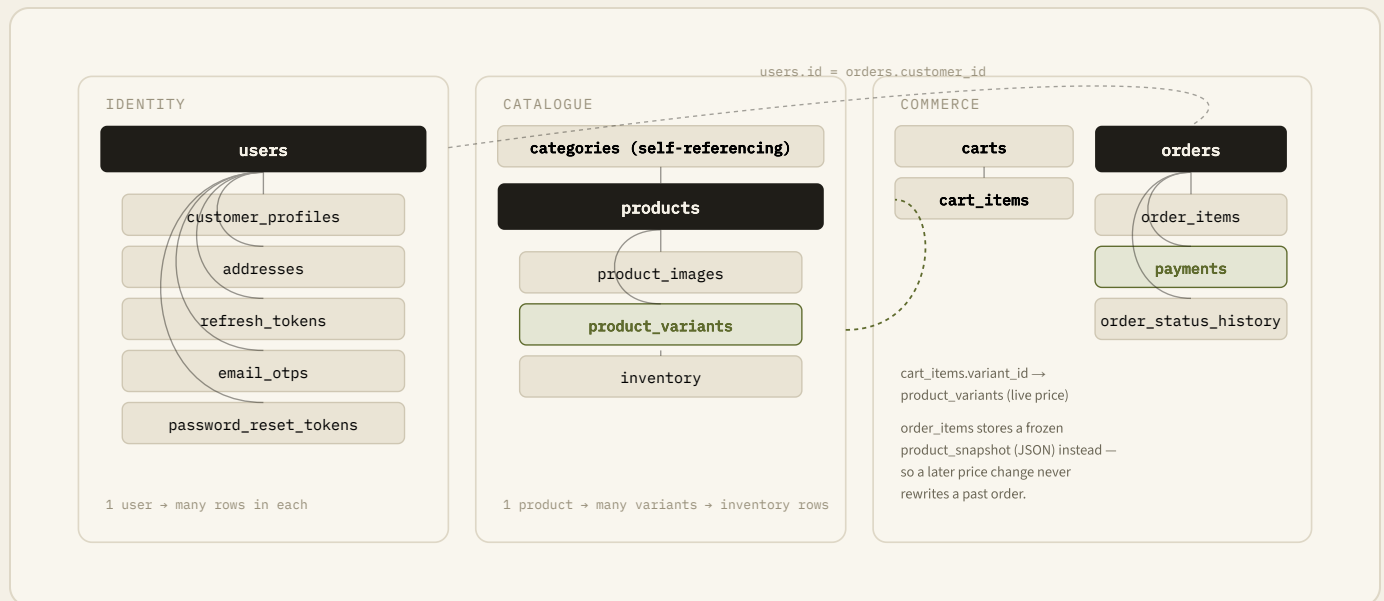
Catalogue pages are React Server Components that fetch straight from the API at request time — good for SEO, no auth involved. Anything personal is a client component that goes through a shared request helper carrying the bearer token and the guest-cart cookie.



`components/Header.tsx` reads both stores directly, so the bag count and account link update instantly on every route — including server-rendered pages — without a page reload.

20 tables, three clusters

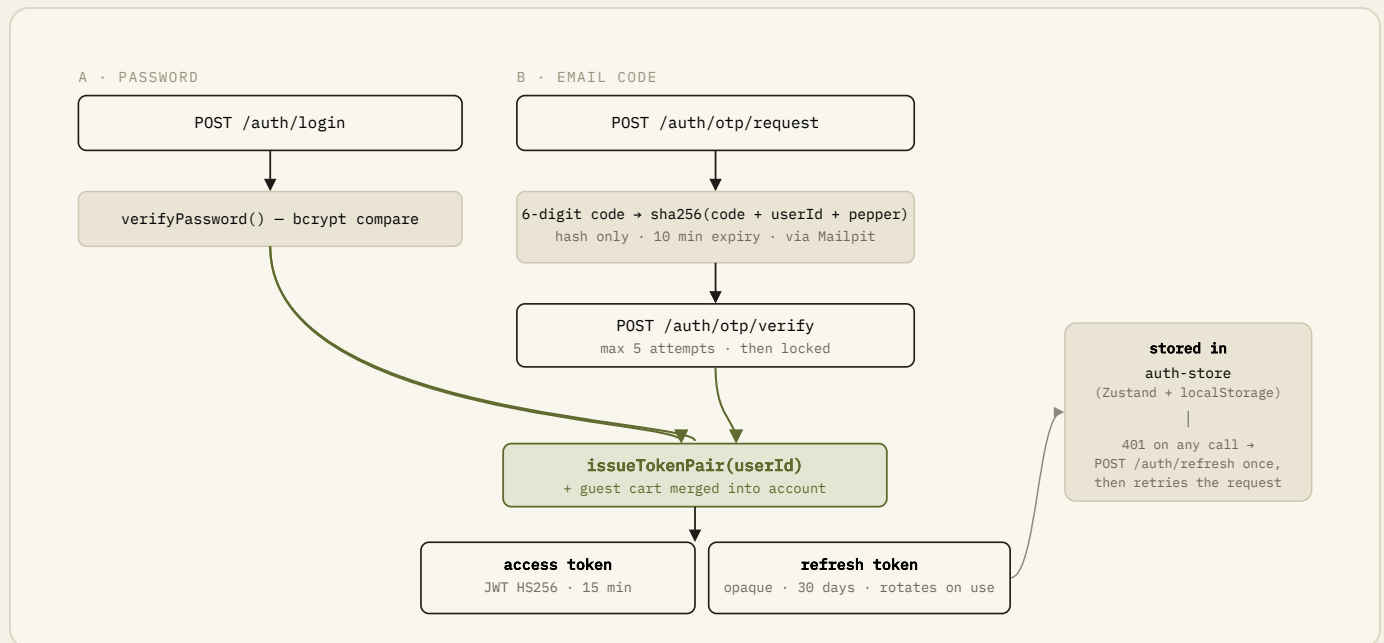
Every ID is a UUID. Money is NUMERIC (12, 2), never a float. The full schema lives in `backend/src/db/schema/*.ts` — this groups it by the part of the product it serves.



`inventory.qty_available` is a Postgres generated column (`qty_on_hand - qty_reserved`) — checkout reads it directly instead of computing availability in application code.

Signing in: two entry paths, one token issuer

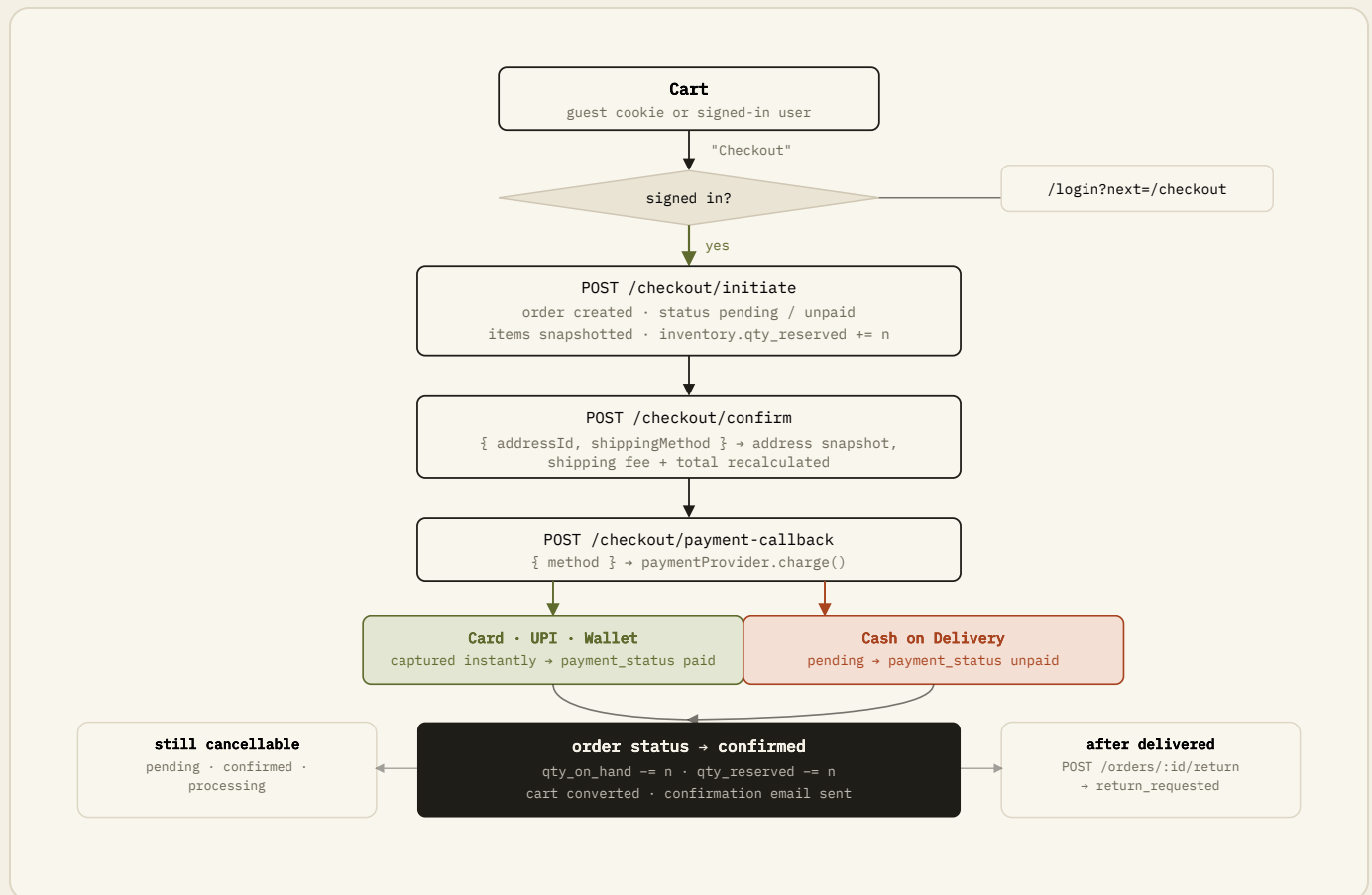
Password and email-code login are separate routes that do separate checks, then both hand off to the exact same token issuer — so a session created either way behaves identically afterwards.



Both request handlers respond identically whether or not the email is registered — no branch in the API response reveals which — the only thing that differs is whether an email actually gets sent.

Cart to confirmed order

Five calls, one state machine. Stock is reserved the moment checkout starts and only released — never overbooked — if the shopper abandons before paying.



Every increment/decrement on `inventory` uses a SQL-level `column = column + n` update rather than read-then-write, so two shoppers checking out the same low-stock variant at once can't both succeed.

⚠ **Known simplification:** reserved stock from an abandoned checkout (initiated, never paid) is not automatically released on a timer yet — the blueprint calls for a 15-minute reservation expiry; Phase 1 doesn't run that sweep.

API surface — /v1

Every endpoint the website calls, grouped by module. `AUTH` means the request needs a valid bearer token.

AUTH

POST	/auth/register	Create account, issue tokens, merge guest cart
POST	/auth/login	Email + password → tokens
POST	/auth/otp/request	Email a 6-digit sign-in code
POST	/auth/otp/verify	Code → tokens
POST	/auth/refresh	Rotate refresh token → new pair
POST	/auth/logout	Revoke one refresh token
POST	/auth/forgot-password	Email a reset link
POST	/auth/reset-password	Token → new password, all sessions revoked

CUSTOMERS

GET	/customers/me	Current profile	AUTH
PUT	/customers/me	Update name / phone / avatar	AUTH
GET	/customers/me/addresses	Saved addresses	AUTH
POST	/customers/me/addresses	Add address	AUTH
PUT	/customers/me/addresses/:id	Update address	AUTH
DEL	/customers/me/addresses/:id	Remove address	AUTH

CATALOGUE

GET	/categories	Full category tree
GET	/products	List · filter by category/price · sort
GET	/products/featured	Homepage picks
GET	/products/search?q=	Name search (ILIKE)
GET	/products/:slug	Detail — variants, stock, related
GET	/categories/:slug/products	Products in a category (+ children)

CART

GET	/cart	Current cart — guest cookie or account
POST	/cart/items	Add variant
PUT	/cart/items/:id	Change quantity
DEL	/cart/items/:id	Remove item
POST	/cart/coupon	Apply coupon code
DEL	/cart/coupon	Remove coupon

CHECKOUT

GET	/checkout/shipping-rates	Standard / Express quotes	
POST	/checkout/initiate	Reserve stock, create pending order	AUTH
POST	/checkout/confirm	Set address + shipping method	AUTH
POST	/checkout/payment-callback	Charge → confirm order	AUTH

ORDERS

GET	/orders	Order history	AUTH
GET	/orders/:id	Detail — items, history	AUTH
POST	/orders/:id/cancel	Cancel (if still eligible)	AUTH
POST	/orders/:id/return	Request return (if delivered)	AUTH
GET	/orders/:id/tracking	Status history	AUTH

CONTACT

POST

/contact

Store message + send acknowledgement email

D:\eCommerce · generated from the working codebase, not a spec · Phase 1 – customer website scope